



5-Minute Demo Video — LinuxAI Companion (Prompt Enhancer Widget)

Audience: HR + technical hiring manager

Goal: Show product working + technical depth + ownership

Format: Screen recording with voiceover



Pre-recording checklist

- [] Open project in VS Code: `code /home/ujjwal/Documents/My\ Apps/apps/widget/linuxai-companion`
- [] Open terminal in same folder for `git log demo`
- [] Launch the widget app: `python3 src/main.py` (have it ready, just minimized)
- [] Open a text editor / ChatGPT tab side-by-side for the Magic Paste demo
- [] Close noisy notifications / Slack / WhatsApp web
- [] Test mic levels — record 10s, listen back
- [] Phone tripod / steady mount; landscape orientation
- [] Mention you'll send the link in chat after — keep video unlisted on YouTube/Loom



Minute-by-minute script

0:00 – 0:25 — Hook + problem (25s)

Show on screen: Yourself / desktop wallpaper

"Hi, I'm Ujjawal Shrivastav. I build full-stack products end-to-end. In the next five minutes I'll walk you through one I shipped solo — a Linux desktop AI companion that lets you rewrite, translate, and enhance text from any application without context-switching. Let me show you the problem it solves first."

Quick visual: Open ChatGPT → copy a prompt → paste into ChatGPT → wait → copy back → paste in app. (2 seconds, just to set up the pain.)

"Every prompt today costs you four window switches. My widget collapses that to zero."

0:25 – 1:30 — Live product demo (65s)

Show on screen: Floating widget icon on desktop edge

Step 1 — Launch + expand (15s)

- Click the floating icon → panel expands

"Single floating shell, GTK4. It's draggable, pinnable, autostart-aware."

Step 2 — Prompt enhancement (20s)

- Paste: `make landing page for saas`

- Click mode dropdown → show the 5 modes (General / Code / Email / Creative / Academic)

- Pick **Code**, click Enhance

- Show the output blossom: structured prompt with role, constraints, output format

"Five context-aware modes — same input gets restructured differently based on intent."

Step 3 — Magic Paste (15s)

- Click Magic Paste
- Switch to a text editor / ChatGPT input — show enhanced text already typed in

"It captured the previously focused window via X11/Wayland window manager and pasted via ydotool. No clipboard pollution."

Step 4 — History (10s)

- Click History tab → show searchable list of past prompts

"Every interaction is logged in SQLite. Search by content, re-run, or copy back."

Step 5 — Subtitle mode (5s flash)

- Toggle subtitle overlay on

"Bonus: it also captures system audio via PipeWire, runs it through Whisper, and overlays real-time translated subtitles. Useful for non-English videos."

1:30 – 2:30 — Architecture walkthrough (60s)

Show on screen: VS Code with `src/` folder open in sidebar

"Architecturally, I split this into five layers — the discipline matters because the project crossed seven thousand lines."

Click each folder briefly (8–10s each):

1. `src/core/` — "Domain logic. Prompt engine, rewrite engine, history store, configuration. No UI, no system calls — testable in isolation."
2. `src/providers/` — "All external services behind abstractions. AI providers — OpenAI, Anthropic, Gemini, and Ollama for offline — share a common interface. Same for STT (Whisper) and translation (Google + Argos). Swapping providers is a one-line change."
3. `src/system/` — "Linux integration layer. Detects X11 vs Wayland, handles clipboard via wl-copy or xclip, simulates paste via uinput, manages the keyring for API keys, captures audio via PipeWire."
4. `src/ui/` — "GTK4 widgets. Each panel is its own component — widget shell, prompt panel, history, settings, setup wizard. UI doesn't talk to providers directly; it goes through core."
5. `src/utils/` — "Constants, logger. Tiny."

"The dependency direction is one-way: UI → core → providers + system. Nothing reaches back the other way."

2:30 – 3:30 — Key decisions (60s)

Show on screen: Open `src/providers/base_provider.py` then `src/system/keyring_manager.py`

Decision 1 — Provider abstraction (20s)

"I knew users would want choice — privacy-conscious folks prefer local Ollama, others want Claude or GPT-4. Instead of branching the codebase, I wrote a single base interface with `enhance(prompt, mode) → string`. Each provider is forty lines, testable independently, and the rest of the app doesn't care which one is active."

Decision 2 — Keyring over .env (20s)

- Open `keyring_manager.py`

"API keys never touch a config file. They live in libsecret — the system keyring backed by gnome-keyring. Even if someone screenshots my config TOML, the keys aren't in it."

Decision 3 — Wayland-first, X11 fallback (20s)

- Open `display_server.py`

"Wayland is the default on modern Ubuntu. But corporate users still run X11. The system layer detects the active server and picks the right tool — wl-clipboard or xclip, ydotool or xdotool. The app code just calls `clipboard.copy()` — implementation switches under the hood."

3:30 – 4:15 — Code quality + commits (45s)

Show on screen: Terminal with `git log --oneline`

```
git log --oneline --reverse
```

Output shows 8 commits — read off in 5 seconds:

"Eight commits across the project lifecycle — scaffold, configuration, providers, engines, system layer, UI, subtitle pipeline, polish. Each commit is a coherent layer, not a 'fixed typo' kind of history."

Show on screen: `pyproject.toml`

"Standard PEP 621 packaging, optional extras for STT and translation so users only install what they use, Ruff for linting with a 100-character line limit. Python 3.12 minimum — modern type hints throughout."

Run quick:

```
find src -name '*.py' | xargs wc -l | tail -1
```

"Forty-six modules, just under seven and a half thousand lines. No file over four hundred lines — strict separation of concerns."

4:15 – 5:00 — Closing pitch (45s)

Show on screen: Back to your face / desktop

"Three takeaways. One: I shipped a real product solo — design, architecture, system-level integration, four AI providers, multi-language pipeline. Two: I make architectural choices deliberately — abstractions when they pay off, KISS when they don't. Three: I'm comfortable across the stack — Python desktop here, but my GitHub also has Next.js, Spring Boot, and FastAPI shipped products."

"Code is in my GitHub at github.com/Shrivaujjawal321. Portfolio with live demos at ujjawal-shrivastav.vercel.app. Excited to talk about how I'd contribute on your team."

"Thanks for watching."

Delivery tips

Tip	Why
Speak slower than feels natural	Five minutes feels long until you watch yourself rush. Aim for 130 words/min.
One thought per breath	Don't string clauses with "and... and... and". Stop, breathe, next sentence.
Click slowly during architecture	Viewers need 1.5s to register what they see. Don't speed-click folders.
Eye contact in opening + closing	Even if you read the rest from notes, the bookends look at the camera.
Don't apologize	No "sorry for my accent" or "this might be rough". Confident frame only.
One take is fine if it lands	Don't hunt for perfect. Land the message. Re-record only if the audio breaks.

Editing checklist

- ☐ Trim dead air from start and end
- ☐ Cut obvious mistakes (only if jarring; small ums are human and fine)
- ☐ Add subtle background music at low volume during the architecture walkthrough (optional)
- ☐ Title card 1s: "LinuxAI Companion — Ujjawal Shrivastav"
- ☐ End card 3s: GitHub + Portfolio URLs

Where to host

- **Loom** (easiest, tracks who watched, free 5-min limit)
- **YouTube unlisted** (no time limit, professional)
- **Google Drive shared link** (zero polish, fastest)

Send the link in your application email or LinkedIn message — never as an attachment.